

Lista de Exercícios

Formatos de Instrução RISC-V: Montagem e Desmontagem

98800-04 – Fundamentos de Sistemas Computacionais · Prof. Iaçã Ianiski Weber

1. Para cada palavra de 32 bits abaixo, identifique o **opcode** em binário, o **formato** (R, I, S, B, U ou J) e o **nome** da instrução. Não é preciso desmontar a instrução inteira: basta olhar o **opcode** e, quando necessário, o **funct3** e o **funct7**.

- | | | | |
|----------------|---------------|---------------|---------------|
| a) 0x00C58533 | c) 0x00A12423 | e) 0x000115B7 | g) 0x00028067 |
| b) 0xFFFF28293 | d) 0xFE031CE3 | f) 0x00C000EF | h) 0x0006A803 |

2. Monte cada instrução abaixo: escreva os **campos em binário** (na ordem do formato correspondente) e a **palavra final em hexadecimal**:

- | | |
|--------------------|-------------------|
| a) add x5, x6, x7 | d) sw s1, 12(s0) |
| b) addi t2, t2, -4 | e) xor a1, a1, a1 |
| c) lw a0, 8(sp) | f) srai t0, t1, 2 |

Para o item (e): o que essa instrução faz com a1?

3. Desmonte cada palavra abaixo para assembly (com nomes ABI dos registradores e imediatos em decimal):

- | | |
|---------------|---------------|
| a) 0x40B50533 | d) 0xFE112E23 |
| b) 0x00F37313 | e) 0x00050593 |
| c) 0xFFC62A83 | |

O item (d) é comum em prólogos de função: o que ele está fazendo? E o item (e) corresponde a qual **pseudoinstrução**?

4. Cada instrução abaixo carrega um imediato que corresponde a um código **ASCII**. Desmonte as quatro instruções, extraia os imediatos e determine a palavra formada pelos caracteres correspondentes:

- | | | | |
|---------------|---------------|---------------|---------------|
| a) 0x05200513 | b) 0x05600593 | c) 0x03300613 | d) 0x03200693 |
|---------------|---------------|---------------|---------------|

Dica

ASCII: 'A' = 65 ... 'Z' = 90; '0' = 48 ... '9' = 57.

5. O trecho abaixo procura o primeiro zero em um vetor de palavras. Os endereços de cada instrução estão indicados:

```

0x100 Loop:  lw  t0, 0(a0)
0x104      beq  t0, zero, Achei
0x108      addi a0, a0, 4
0x10C      jal  zero, Loop
0x110 Achei:  ...

```

- Calcule o deslocamento (em bytes) do **beq** e monte a instrução completa: mostre os campos **imm[12|10:5]**, **rs2**, **rs1**, **funct3**, **imm[4:1|11]** e **opcode**, e o valor final em hexadecimal.
 - Faça o mesmo para o **jal zero, Loop** (deslocamento **negativo**): campos **imm[20|10:1|11|19:12]**, **rd**, **opcode** e o hexadecimal final.
 - Qual o alcance máximo de um **beq**, em instruções de 32 bits? E de um **jal**?
 - Suponha agora que o rótulo **Achei** esteja a 3000 instruções de distância. O **beq** ainda alcança? Se não, reescreva o desvio condicional usando duas instruções, conforme visto em aula.
6. A rotina abaixo zera a1 palavras de memória a partir do endereço em a0. Durante a operação em um ambiente sujeito a radiação, **exatamente um bit** de **uma** das palavras foi invertido (*bit flip*) e a rotina parou de funcionar:

Endereço	Conteúdo
0x00	0x00053023
0x04	0x00450513
0x08	0xFFFF58593
0x0C	0xFE059AE3
0x10	0x00008067

- a) Desmonte as cinco palavras. Uma delas não corresponde a nenhuma instrução RV32I válida. Qual, e por quê?
- b) Qual bit (indique a posição, 0–31) foi invertido? Qual era a palavra original e que instrução ela codificava?
- c) Com a correção aplicada, reescreva a rotina completa em assembly com rótulos.

Curiosidade

Bit flips por radiação são um problema real em satélites e até em servidores na superfície. Por isso, memórias de sistemas críticos usam códigos de correção de erros (ECC), e decodificadores de instrução detectam *opcodes* inválidos e disparam uma exceção.

7. A palavra 0x00008067 aparece milhares de vezes em praticamente qualquer binário RISC-V compilado.
 - a) Desmonte-a. Qual instrução é? A qual pseudoinstrução corresponde?
 - b) Por que ela aparece no final de quase toda função?
 - c) Monte a instrução `jal ra, Func`, onde `Func` está 32 bytes *adiante*. Dê o hexadecimal final.
 - d) Diferentemente de B e J, o imediato do `jalr` **não** é multiplicado por 2. Explique por quê.
8. Constantes de 32 bits são construídas com o par `lui + addi`.
 - a) Escreva a sequência `lui/addi` que coloca 0x12345678 em `t0`.
 - b) Repita para 0xDEADBEEF em `t1`. Mostre por que a abordagem direta (`lui` com 0xDEADB) produz um resultado incorreto e apresente a sequência correta.
 - c) Explique a regra geral: quando é preciso ajustar o imediato do `lui`, e por quê?
9. As instruções da tabela abaixo foram montadas à mão e algumas contêm erros. Para cada linha, verifique se o hexadecimal realmente codifica a instrução pretendida. Se estiver **errado**, desmonte o que foi *de fato* codificado, explique o erro cometido e dê o hexadecimal correto.

#	Instrução pretendida	Hex montado
1	<code>sub s0, s1, s2</code>	0x40990433
2	<code>sw t0, 20(sp)</code>	0x28512023
3	<code>beq a0, a1, +8</code> (pular para 2 instruções adiante)	0x00B50163
4	<code>lhu t2, 6(a0)</code>	0x00655383

10. O *dump* de memória abaixo contém um programa cujo código-fonte não está disponível:

Endereço	Conteúdo
0x200	0x00000293
0x204	0x00058863
0x208	0x00A282B3
0x20C	0xFFFF58593
0x210	0xFF5FF06F
0x214	0x00028513

- a) Desmonte as seis palavras (atenção aos deslocamentos das instruções de desvio).
 - b) Reescreva o programa em assembly com rótulos legíveis.
 - c) O que esse programa calcula, considerando que recebe dois valores em `a0` e `a1` e devolve o resultado em `a0`?
 - d) Se o programa for chamado com `a0 = 6` e `a1 = 7`, qual o valor final de `a0`?
 - e) Quantas vezes a instrução no endereço 0x204 é executada nesse caso?
11. Justifique tecnicamente, em 1–2 frases cada, as seguintes decisões de projeto do RISC-V:
 - a) Por que **rs1**, **rs2** e **rd** ficam **sempre nas mesmas posições** em todos os formatos que os utilizam?
 - b) Por que o formato S **divide o imediato** em dois pedaços em vez de mantê-lo contíguo como no formato I?
 - c) Por que os deslocamentos de B e J são codificados em **múltiplos de 2 bytes**, e não de 1 ou de 4?